Arquitectura de software de Prosta-AI

Yeison Bolaños Ruiz

Juan David Riascos Erazo

Sergio Gabriel Meneses Silva

Yimis Buenaventura

Fernando José Angulo Núñez

Facultad de Ingeniería, Universidad Santiago de Cali

Programa de Ingeniería de Sistemas

Gestión de Proyectos de TI

Jair Enrique Sanclemente Castro

22 de octubre de 2026

Definición y propósito

Este documento aplica ese enfoque a Prosta-AI. Con la tabla de casos de uso se puede revisar que los componentes cubren los requisitos de la matriz de trazabilidad; la comparación de alternativas justifica el estilo elegido antes de construir, y el control de factibilidad atiende desde ya el riesgo 1 del Acta. La vista de componentes es, además, la referencia común para el equipo y el docente. El documento corresponde al paquete 1.3.2.4 de la EDT (arquitectura de software) y a las actividades 1306 a 1309 del Ciclo 2. Se queda en el nivel arquitectónico: los esquemas de tablas, las rutas, los algoritmos y los protocolos de pruebas van en entregables posteriores.

Prosta-AI es una aplicación web académica para cargar estudios de imagen desidentificados, segmentar la próstata, visualizar una región de interés (ROI) y, cuando hay una máscara de referencia, obtener métricas de evaluación. No da diagnósticos, clasificaciones clínicas ni recomendaciones terapéuticas.

Alternativas y decisión arquitectónica

Se adopta una arquitectura cliente-servidor por capas, con React en la presentación y un backend modular en FastAPI y Python. PostgreSQL guarda la información de los trabajos, y un almacenamiento privado contiene los estudios, las máscaras y los pesos del modelo. La inferencia usa PyTorch y MONAI. La U-Net 3D es el modelo neuronal; el estilo arquitectónico de la aplicación es otra cosa.

Tabla 1

Comparación de alternativas

Alternativa	Ventaja principal	Limitación y decisión
Monolito modular, un proceso	Cuesta menos integrarlo y desplegarlo, y alcanza para integrar el flujo completo en el Ciclo 6.	La inferencia puede competir con la API por CPU y memoria. **Elegida como mínimo viable.**
Backend con ejecutor separado	Aísla la carga pesada y responde mejor cuando hay varios trabajos.	Exige coordinación, recuperación y un despliegue más complejo. **Se adopta solo si las mediciones lo justifican.**

Alternativa	Ventaja principal	Limitación y decisión
Microservicios	Cada componente puede escalar por separado.	Su complejidad operativa es excesiva para un alcance académico. **Descartada.**

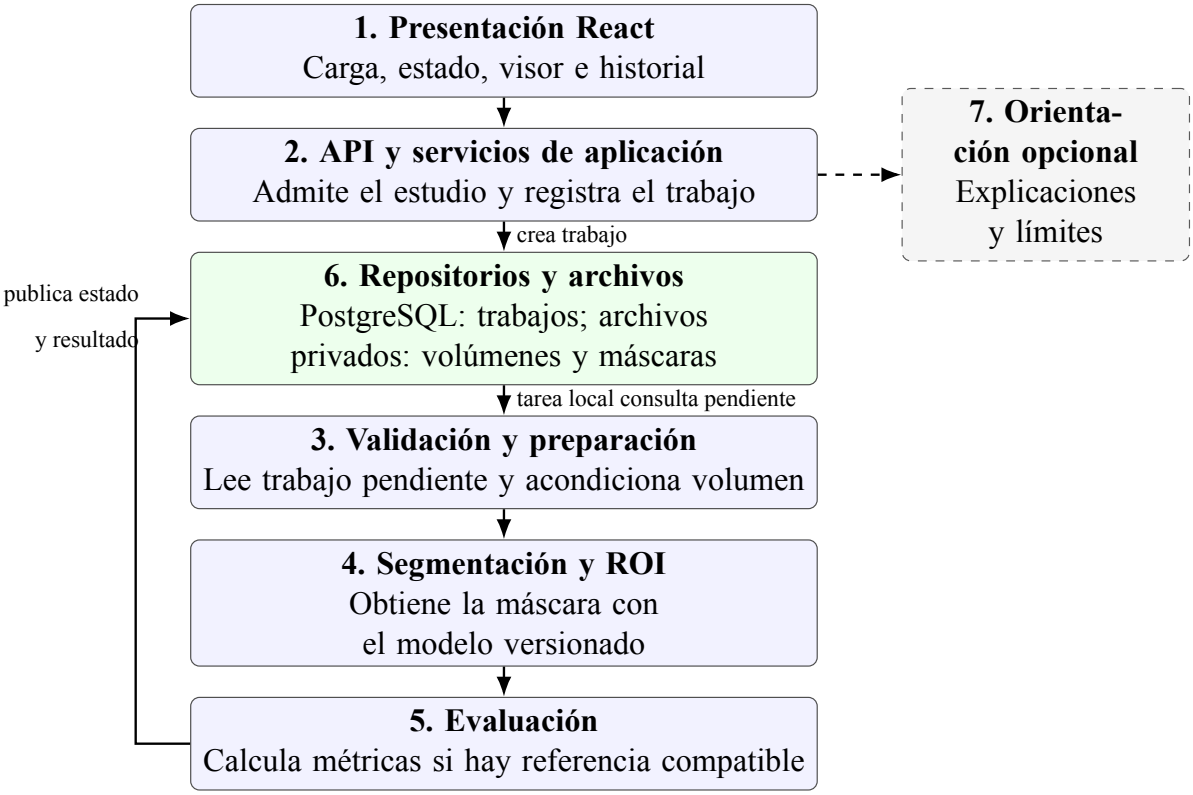
Aunque el despliegue inicial sea un solo proceso, el código se mantiene separado en módulos. La decisión se puede revisar si las mediciones muestran que la interfaz se bloquea o que los tiempos no alcanzan el objetivo de rendimiento.

Vista lógica de componentes

La Figura 1 y la Tabla 2 describen los mismos siete componentes. La API no llama directamente a la segmentación: comprueba que la carga sea admisible y crea un trabajo pendiente en PostgreSQL. Luego, una tarea local del backend toma ese trabajo a través de los repositorios y ejecuta en orden la validación detallada, la preparación, la segmentación y la evaluación. El resultado se guarda en el almacenamiento, donde la API puede consultarlo.

Figura 1

Vista lógica de los siete componentes de Prosta-AI



Nota. Las flechas indican responsabilidades y el orden lógico del flujo; no son procesos independientes. Si falta una referencia válida, la evaluación se omite y el resultado de la segmentación se conserva.

Tabla 2
Responsabilidades de los componentes

Componente	Responsabilidad arquitectónica
1. Presentación React	Permitir la carga, la consulta del avance y la visualización de cortes y ROI.
2. API y servicios	Coordinar el acceso con nombre de usuario y contraseña, la creación de trabajos y la recuperación de resultados.
3. Validación y preparación	Comprobar los requisitos de entrada, la privacidad y la geometría antes de la inferencia.
4. Segmentación y ROI	Ejecutar el modelo y devolver una máscara alineada con la imagen.
5. Evaluación	Comparar la máscara con una referencia compatible e informar Dice e IoU cuando aplique.
6. Repositorios y archivos	Guardar en PostgreSQL los usuarios (solo alias y contraseña cifrada), los trabajos y sus estados, y custodiar los archivos privados.
7. Orientación opcional	Explicar el uso de la aplicación y sus límites sin tomar decisiones clínicas.

Flujo de procesamiento y casos de uso

El sistema tiene seis casos de uso. En CU02 el usuario carga los estudios, y CU03 empieza cuando pulsa el botón «Procesar». Desde ese momento el sistema hace por sí solo la validación, la preparación, la segmentación y la evaluación, como pide el requisito 4.2.

1. El usuario ingresa con su nombre de usuario y contraseña. Si el nombre no existe, el sistema ofrece crearlo y pide repetir la contraseña; si las dos coinciden, guarda el usuario con la contraseña cifrada.

2. El usuario carga un estudio y el sistema le confirma si fue admitido o rechazado.

3. La API comprueba que el archivo sea DICOM o NIfTI y que esté dentro de los límites básicos, y guarda temporalmente el archivo admitido. Cuando el usuario pulsa «Procesar», la API crea un trabajo pendiente en PostgreSQL.

4. Una tarea local del mismo backend revisa los trabajos pendientes y los procesa de uno en uno: valida el contenido y la privacidad, prepara el volumen, ejecuta la segmentación y pide

la evaluación si hay referencia.

5. El sistema registra si el trabajo terminó o falló. El frontend consulta el avance y muestra la ROI, el aviso experimental y las métricas disponibles.

Si la carga se rechaza en la admisión, no se crea ningún trabajo; si algo falla en la validación posterior, el fallo queda registrado. Cuando no hay referencia, la máscara se muestra igual y las métricas aparecen como no disponibles. Los estados generales son *pendiente*, *en proceso*, *completado* y *fallido*; la recuperación, los mensajes y las transacciones se definirán en el diseño técnico.

Tabla 3
Casos de uso y trazabilidad

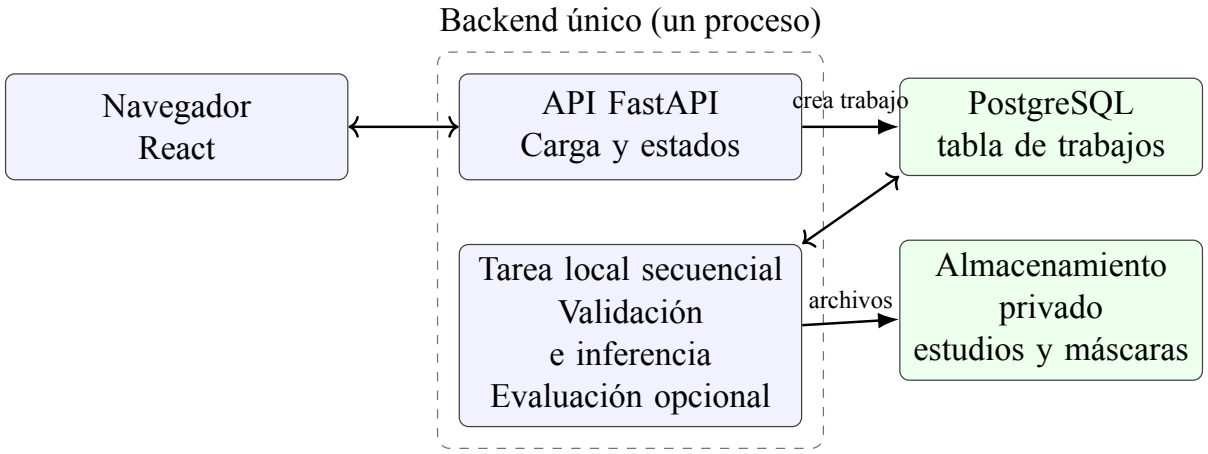
ID	Nombre del caso de uso	Requisitos relacionados
CU01	Acceder a la aplicación	Acceso con nombre de usuario y contraseña; creación de usuario con confirmación de contraseña; usabilidad (4.7, 5.2).
CU02	Cargar estudios de MRI de prueba	Carga, formato y privacidad de entrada (4.1, 2.2).
CU03	Ejecutar el procesamiento automático	Inicio con el botón «Procesar»; preparación y segmentación sin intervención adicional (3.2, 3.4, 4.2, 5.1).
CU04	Visualizar y comparar la ROI	Superposición, explicación y aviso experimental (4.3, 4.4, 2.3).
CU05	Consultar estudios y resultados registrados	Persistencia e historial (4.5).
CU06	Consultar orientación mediante agentes de IA (opcional)	Orientación limitada y revisable (4.6); prioridad Could Have.

La numeración CU01–CU06 es la misma del diagrama de casos de uso, la matriz de trazabilidad y la priorización MoSCoW. CU02 y CU03 se separan porque cargar y empezar el procesamiento son dos acciones distintas del usuario. La orientación de CU06 está fuera del flujo obligatorio.

Vista de despliegue mínimo viable

Para la integración del Ciclo 6 se propone un servidor con un solo proceso de backend, que atiende la API y arranca una tarea local que procesa la tabla de trabajos en orden. React es el cliente web, y el backend depende de PostgreSQL y del almacenamiento privado. No hacen falta un ejecutor distribuido, una cola externa ni microservicios.

Figura 2
Despliegue mínimo para el Ciclo 6



Nota. El modelo se entrena fuera del flujo de uso de la aplicación. Separar el procesador en otro proceso queda como posible mejora posterior; el mínimo viable no lo necesita.

Atributos de calidad, factibilidad y riesgo

La arquitectura da prioridad a la privacidad de los estudios, la trazabilidad de los resultados, la mantenibilidad y un recorrido fácil de entender desde la carga hasta la ROI. El sistema usa fuentes autorizadas y datos desidentificados, y guarda los originales, las máscaras y los pesos en almacenamiento privado. Cada resultado debe poder relacionarse con la versión del modelo y la procedencia del estudio. La calidad de la segmentación se medirá con Dice e IoU en el plan de evaluación, sin tomar esas cifras como certeza diagnóstica.

La meta de rendimiento es tener el resultado en dos minutos (120 segundos), pero todavía no está demostrada. La capacidad de cómputo para la inferencia 3D es el riesgo 1 del Acta, cuya respuesta prevista es usar modelos preentrenados con ajuste ligero, Google Colab gratuito y una

resolución de trabajo menor. Antes de ampliar la arquitectura hay que hacer una medición temprana con un volumen representativo, pesos reales y el hardware previsto, y registrar por separado el tiempo de preparación, el de inferencia y el del recorrido completo hasta la visualización. Esa evidencia incluye también el tamaño de los volúmenes y la memoria disponible.

Tabla 4

Control de factibilidad en la entrega

Prioridad	**Acción inmediata**	**Decisión según resultado**
Riesgo 1: tiempo de inferencia	Ciclo 3 (actividad 1408): medir CPU, GPU y memoria con un estudio representativo y comparar con 120 s.	Si supera la meta: usar Google Colab, un modelo preentrenado más ligero o menor resolución; si aun así no se cumple, documentar la limitación.
Disponibilidad de la API	Comprobar que la interfaz puede consultar el estado mientras corre la tarea local.	Si se bloquea, separar el procesador en una etapa posterior.
Privacidad e integridad	Comprobar que se rechazan los datos no aptos y que el resultado permitido se recupera.	Ajustar la validación, el aislamiento y la trazabilidad sin ampliar el alcance funcional.

La primera medición del tiempo de inferencia se hace en el Ciclo 3 (actividad 1408). El mínimo viable integrado del Ciclo 6 incluye la ruta de carga, la tabla de trabajos, el procesamiento local secuencial y el registro del estado (actividades 1701 a 1706). La ROI se desarrolla en el Ciclo 4 y la evaluación formal en el Ciclo 5. Las estructuras complejas de concurrencia o recuperación solo se construirán si hay evidencia de que hacen falta.

Supuestos y decisiones pendientes

Se asume que se usarán estudios públicos desidentificados, acceso con nombre de usuario y contraseña sin roles ni datos personales, y un modelo de segmentación 3D que funcione con los recursos del equipo. Se aceptan estudios en DICOM y NIfTI; en los DICOM, la validación elimina los metadatos que puedan identificar a una persona o rechaza el archivo (requisito 2.2). Faltan por confirmar los límites de tamaño, el conjunto de datos y la disponibilidad de GPU. Tanto la meta de 120 segundos como la posible necesidad de ejecutar la inferencia en otro proceso dependen de la prueba temprana del riesgo 1.

Referencias

Bass, L., Clements, P., & Kazman, R. (2013). *Software architecture in practice* (3rd ed.). Addison-Wesley.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.